

AG35-CET QuecOpen(SDK) Wi-Fi 开发指导

LTE Standard 模块系列

版本：1.0.0

日期：2023-11-13

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-11-13	Sunshine HUANG	文档创建
1.0.0	2023-11-13	Sunshine HUANG	临时版本

目录

文档历史	3
目录	4
表格索引	7
1 引言	8
2 Wi-Fi 功能实现	9
2.1. Wi-Fi 工作模式	9
2.2. Wi-Fi P2P 简介	9
3 Wi-Fi API	11
3.1. 头文件	11
3.2. 函数概览	11
3.3. 函数详解	14
3.3.1. ql_wifi_init	14
3.3.2. ql_wifi_deinit	15
3.3.3. ql_wifi_enable	15
3.3.4. ql_wifi_disable	15
3.3.5. ql_wifi_set_enable_status_ind_cb	16
3.3.5.1. ql_wifi_enable_status_ind_cb_f	16
3.3.5.2. QL_WIFI_ENABLE_STATUS_E	17
3.3.6. ql_wifi_work_mode_set	17
3.3.6.1. QL_WIFI_WORK_MODE_E	18
3.3.7. ql_wifi_set_bridge	18
3.3.8. ql_wifi_work_mode_get	19
3.3.9. ql_wifi_ap_ssid_set	19
3.3.9.1. QL_WIFI_AP_INDEX_E	20
3.3.10. ql_wifi_ap_ssid_get	20
3.3.11. ql_wifi_ap_ssid_hidden_set	21
3.3.12. ql_wifi_ap_ssid_hidden_get	21
3.3.13. ql_wifi_ap_mode_set	22
3.3.13.1. QL_WIFI_AP_MODE_TYPE_E	23
3.3.14. ql_wifi_ap_mode_get	23
3.3.15. ql_wifi_ap_bandwidth_set	24
3.3.15.1. QL_WIFI_BANDWIDTH_E	24
3.3.16. ql_wifi_ap_bandwidth_get	25
3.3.17. ql_wifi_ap_channel_set	25
3.3.18. ql_wifi_ap_channel_get	26
3.3.19. ql_wifi_ap_max_sta_num_set	27
3.3.20. ql_wifi_ap_max_sta_num_get	27
3.3.21. ql_wifi_ap_country_code_set	28
3.3.22. ql_wifi_ap_country_code_get	28
3.3.23. ql_wifi_ap_isolate_set	29

3.3.24.	ql_wifi_ap_isolate_get.....	30
3.3.25.	ql_wifi_ap_mac_acl_rule_set.....	30
3.3.25.1.	QL_WIFI_MAC_ACL_RULE_E.....	31
3.3.26.	ql_wifi_ap_mac_acl_rule_get.....	31
3.3.27.	ql_wifi_ap_acl_mac_set.....	32
3.3.27.1.	QL_WIFI_ACL_MAC_CMD_E.....	32
3.3.28.	ql_wifi_ap_acl_mac_get.....	33
3.3.28.1.	ql_wifi_acl_mac_list_t.....	33
3.3.28.2.	ql_wifi_mac_addr_t.....	34
3.3.29.	ql_wifi_ap_auth_set.....	34
3.3.29.1.	ql_wifi_ap_auth_t.....	35
3.3.29.2.	QL_WIFI_AUTH_E.....	36
3.3.29.3.	QL_WIFI_AUTH_WPA_PSK_E.....	37
3.3.30.	ql_wifi_ap_auth_get.....	37
3.3.31.	ql_wifi_ap_set_status_ind_cb.....	38
3.3.31.1.	ql_wifi_ap_status_ind_cb_f.....	38
3.3.31.2.	QL_WIFI_AP_STATUS_E.....	39
3.3.31.3.	ql_wifi_ap_status_t.....	40
3.3.32.	ql_wifi_ap_start.....	40
3.3.33.	ql_wifi_ap_stop.....	41
3.3.34.	ql_wifi_ap_get_status.....	41
3.3.35.	ql_wifi_sta_ssid_set.....	42
3.3.36.	ql_wifi_sta_ssid_get.....	42
3.3.37.	ql_wifi_sta_auth_set.....	43
3.3.37.1.	ql_wifi_sta_auth_t.....	43
3.3.38.	ql_wifi_sta_auth_get.....	44
3.3.39.	ql_wifi_sta_set_status_ind_cb.....	44
3.3.39.1.	ql_wifi_sta_status_ind_cb_f.....	45
3.3.39.2.	QL_WIFI_STA_STATUS_E.....	45
3.3.39.3.	ql_wifi_sta_status_t.....	46
3.3.40.	ql_wifi_sta_start.....	47
3.3.41.	ql_wifi_sta_stop.....	48
3.3.42.	ql_wifi_sta_get_status.....	48
3.3.43.	ql_wifi_sta_start_scan.....	49
3.3.44.	ql_wifi_sta_set_scan_result_ind_cb.....	49
3.3.44.1.	ql_wifi_sta_scan_result_ind_cb_f.....	50
3.3.44.2.	ql_wifi_sta_scan_result_t.....	50
3.3.44.3.	ql_wifi_sta_scan_list_t.....	50
3.3.44.4.	ql_wifi_sta_scan_info_t.....	51
3.3.44.5.	QL_WIFI_SCAN_REASON_CODE_E.....	52
3.3.45.	ql_wifi_set_service_error_cb.....	52
3.3.45.1.	ql_wifi_service_error_cb_f.....	53
3.3.46.	ql_wifi_set_ap_sta_connect_ind_cb.....	53
3.3.46.1.	ql_wifi_ap_sta_connect_ind_cb_f.....	54

3.3.46.2.	ql_wifi_sta_connect_status_t.....	54
3.3.47.	ql_wifi_country_code_set.....	55
3.3.48.	ql_wifi_country_code_get	55
3.3.49.	ql_wifi_p2p_dev_name_set	56
3.3.50.	ql_wifi_p2p_dev_name_get	56
3.3.51.	ql_wifi_p2p_dev_type_set.....	57
3.3.52.	ql_wifi_p2p_dev_type_get	57
3.3.53.	ql_wifi_p2p_oper_class_channel_set.....	58
3.3.54.	ql_wifi_p2p_oper_class_channel_get.....	59
3.3.55.	ql_wifi_p2p_ssid_postfix_set	59
3.3.56.	ql_wifi_p2p_ssid_postfix_get	60
3.3.57.	ql_wifi_p2p_set_enable_status_ind_cb.....	61
3.3.57.1.	ql_wifi_p2p_enable_status_ind_cb_f.....	61
3.3.57.2.	QL_WIFI_P2P_ENABLE_STATUS_E	61
3.3.58.	ql_wifi_p2p_enable	62
3.3.59.	ql_wifi_p2p_disable.....	62
3.3.60.	ql_wifi_p2p_set_dev_found_ind_cb.....	63
3.3.60.1.	ql_wifi_p2p_dev_found_ind_cb_f.....	63
3.3.60.2.	ql_wifi_p2p_found_dev_info_t	64
3.3.61.	ql_wifi_p2p_dev_find_start	64
3.3.62.	ql_wifi_p2p_dev_find_stop.....	65
3.3.63.	ql_wifi_p2p_get_status.....	65
3.3.63.1.	ql_wifi_p2p_status_t.....	66
3.3.63.2.	QL_WIFI_P2P_STATUS_E.....	67
3.3.63.3.	QL_WIFI_P2P_ROLE_E.....	68
3.3.64.	ql_wifi_p2p_set_status_ind_cb.....	68
3.3.64.1.	ql_wifi_p2p_status_ind_cb_f.....	69
3.3.65.	ql_wifi_p2p_set_peer_dev_req_ind_cb.....	69
3.3.65.1.	ql_wifi_p2p_peer_dev_req_ind_cb_f.....	70
3.3.65.2.	ql_wifi_p2p_req_peer_dev_info_t.....	70
3.3.66.	ql_wifi_p2p_connect.....	70
3.3.67.	ql_wifi_p2p_disconnect.....	71
4	示例	72
4.1.	设置 STA 模式	72
4.2.	设置单 AP 模式.....	74
4.3.	Wi-Fi P2P 模式	76
5	附录 参考文档及术语缩写	79

表格索引

表 1: 函数概览	11
表 2: 参考文档	79
表 3: 术语缩写	79

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 模块支持 QuecOpen[®]方案。QuecOpen[®]是基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen[®]的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen[®]方案，主要介绍移远通信 AG35-CET QuecOpen[®]模块 Wi-Fi 功能相关的 API 及其实例。

Preliminary
Quectel Confidential

2 Wi-Fi 功能实现

AG35-CET 模块通过搭配移远通信 Wi-Fi 模块 FC30R 可实现 Wi-Fi 功能。本章主要介绍所述 Wi-Fi 模块支持的工作模式，以及 Wi-Fi P2P 功能。

2.1. Wi-Fi 工作模式

移远通信 Wi-Fi 模块 FC30R 支持两种工作模式，分别为：

1) AP 模式

AP 模式下，模块仅开放一个热点，支持 2.4 GHz 频段。

2) STA 模式

STA 模式下，模块作为一个无线网卡使用，能够连接 2.4 GHz 频段的热点。

2.2. Wi-Fi P2P 简介

Wi-Fi 模块 FC30R 还支持 Wi-Fi 直连功能，即 Wi-Fi P2P 功能。

Wi-Fi Direct（Wi-Fi 直连）是 Wi-Fi 联盟推出的一项重要协议规范，Wi-Fi P2P 是该协议规范中的一种连接方式。Wi-Fi Direct（Wi-Fi 直连）支持多个 Wi-Fi 设备在无 AP 的场景下相互连接，是 Wi-Fi Display（无线显示）功能的基础。在 Miracast（Wi-Fi 联盟对支持 Wi-Fi Display 功能的设备的认证名称）应用场景中，支持 P2P 的智能手机可直接连接支持 P2P 的 IVI 系统，该智能手机可将本地视频或媒体资源传送至 IVI 系统进行显示或播放。

Wi-Fi P2P 结构中定义了三个组件，即一个设备，两个角色：

P2P Device:

- 支持 P2P Group Owner 和 P2P Client 两个角色；
- 协商成为 P2P Group Owner（GO）或 P2P Client 角色；
- 支持 WSC 和 P2P Discovery 机制；
- 可支持 WLAN 和 P2P 并发操作。

P2P Group Owner (GO) 角色:

- 类似于接入点的实体, 为相互关联的 Clients (P2P Clients 或 Legacy Clients) 提供基础服务集功能和服务;
- 提供 WSC Internal Registrar 功能;
- 可为相互关联的 Clients 彼此之间建立通信;
- 可将与其关联的 Clients 同时接入不同 WLAN。

P2P Client 角色:

- 执行 non-AP STA 功能;
- 提供 WSC Enrollee 功能。

组建 P2P Group (即 P2P Network) 之前, 终端均为独立的 P2P Device。P2P Device 之间完成 P2P 协商后, 有且只有一个 Device 将作为 GO 角色, 其他 P2P Device 来扮演 Client 的角色。

如下图 P2P 网络组织结构所示, 一个 P2P Group 中只有一个 GO, 一个 GO 支持一个或者多个 Client 连接。由于 GO 的功能类似于 AP, 所以周围不支持 P2P 功能的 STA 也能发现并关联到 GO, 这些 STA 称为 Legacy Client。

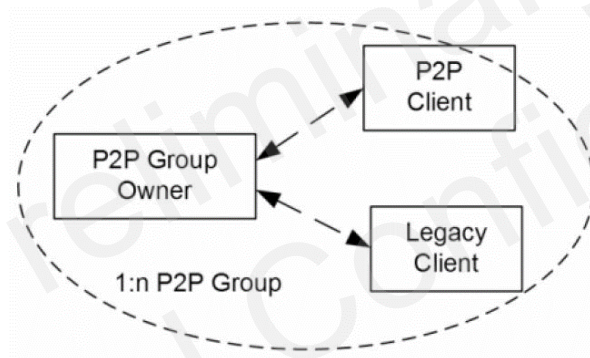


图 1: Wi-Fi P2P 组织结构图

3 Wi-Fi API

3.1. 头文件

Wi-Fi API 头文件为 *ql_wifi.h* 和 *ql_wifi_common.h*，位于 SDK 包的 *ql-sysroots/usr/include/ql-sdk/* 目录下。其中 *ql_wifi.h* 定义了 Wi-Fi 相关的 API，*ql_wifi_common.h* 定义了 Wi-Fi 相关的数据结构。若无特别说明，本文档所涉及头文件均在该目录下。

3.2. 函数概览

表 1：函数概览

函数	说明
<i>ql_wifi_init()</i>	初始化 Wi-Fi 服务
<i>ql_wifi_deinit()</i>	去初始化 Wi-Fi 服务
<i>ql_wifi_enable()</i>	开启 Wi-Fi 功能
<i>ql_wifi_disable()</i>	关闭 Wi-Fi 功能
<i>ql_wifi_set_enable_status_ind_cb()</i>	注册 Wi-Fi 状态回调函数
<i>ql_wifi_work_mode_set()</i>	设置 Wi-Fi 工作模式
<i>ql_wifi_set_bridge()</i>	设置当前 Wi-Fi 加入的网桥
<i>ql_wifi_work_mode_get()</i>	获取当前 Wi-Fi 工作模式
<i>ql_wifi_ap_ssid_set()</i>	设置 AP 模式的 SSID
<i>ql_wifi_ap_ssid_get()</i>	获取 AP 模式的 SSID
<i>ql_wifi_ap_ssid_hidden_set()</i>	设置 AP 模式的 SSID 隐藏状态

<code>ql_wifi_ap_ssid_hidden_get()</code>	获取 AP 模式的 SSID 隐藏状态
<code>ql_wifi_ap_mode_set()</code>	设置 AP 模式的工作协议模式
<code>ql_wifi_ap_mode_get()</code>	获取 AP 模式的工作协议模式
<code>ql_wifi_ap_bandwidth_set()</code>	设置 AP 模式的带宽
<code>ql_wifi_ap_bandwidth_get()</code>	获取 AP 模式的带宽
<code>ql_wifi_ap_channel_set()</code>	设置 AP 模式的信道
<code>ql_wifi_ap_channel_get()</code>	获取 AP 模式的信道
<code>ql_wifi_ap_max_sta_num_set()</code>	设置 AP 模式终端最大连接数量
<code>ql_wifi_ap_max_sta_num_get()</code>	获取 AP 模式终端最大连接数量
<code>ql_wifi_ap_country_code_set()</code>	设置 AP 模式的国家码
<code>ql_wifi_ap_country_code_get()</code>	获取 AP 模式的国家码
<code>ql_wifi_ap_isolate_set()</code>	设置 AP 隔离状态
<code>ql_wifi_ap_isolate_get()</code>	获取 AP 隔离状态
<code>ql_wifi_ap_mac_acl_rule_set()</code>	设置 AP 模式的 MAC 地址访问规则
<code>ql_wifi_ap_mac_acl_rule_get()</code>	获取 AP 模式的 MAC 地址访问规则
<code>ql_wifi_ap_acl_mac_set()</code>	为当前 MAC 地址访问规则添加或者删除指定的 MAC 地址
<code>ql_wifi_ap_acl_mac_get()</code>	获取当前 MAC 地址访问规则中所有的 MAC 地址
<code>ql_wifi_ap_auth_set()</code>	设置 AP 模式的安全认证
<code>ql_wifi_ap_auth_get()</code>	获取 AP 模式的安全认证信息
<code>ql_wifi_ap_set_status_ind_cb()</code>	注册 AP 模式状态回调函数
<code>ql_wifi_ap_start()</code>	开启 AP 功能
<code>ql_wifi_ap_stop()</code>	关闭 AP 功能
<code>ql_wifi_ap_get_status()</code>	获取 AP 模式的状态信息
<code>ql_wifi_sta_ssid_set()</code>	设置 STA 模式的 SSID
<code>ql_wifi_sta_ssid_get()</code>	获取 STA 模式的 SSID

<code>ql_wifi_sta_auth_set()</code>	设置 STA 模式的安全认证
<code>ql_wifi_sta_auth_get()</code>	获取 STA 模式的安全认证信息
<code>ql_wifi_sta_set_status_ind_cb()</code>	注册 STA 模式状态回调函数
<code>ql_wifi_sta_start()</code>	开启 STA 功能
<code>ql_wifi_sta_stop()</code>	关闭 STA 功能
<code>ql_wifi_sta_get_status()</code>	获取 STA 模式的状态信息
<code>ql_wifi_sta_start_scan()</code>	STA 模式扫描外部热点
<code>ql_wifi_sta_set_scan_result_ind_cb()</code>	注册扫描结果回调函数
<code>ql_wifi_set_service_error_cb()</code>	注册 Wi-Fi 服务异常回调函数
<code>ql_wifi_set_ap_sta_connect_ind_cb()</code>	注册 AP 模式和 STA 设备连接状态回调函数
<code>ql_wifi_country_code_set()</code>	设置 Wi-Fi 国家码
<code>ql_wifi_country_code_get()</code>	获取 Wi-Fi 国家码
<code>ql_wifi_p2p_dev_name_set()</code>	设置 Wi-Fi P2P 设备名称
<code>ql_wifi_p2p_dev_name_get()</code>	获取 Wi-Fi P2P 设备名称
<code>ql_wifi_p2p_dev_type_set()</code>	设置 Wi-Fi P2P 设备类型
<code>ql_wifi_p2p_dev_type_get()</code>	获取 Wi-Fi P2P 设备类型
<code>ql_wifi_p2p_oper_class_channel_set()</code>	设置 Wi-Fi P2P 设备的工作频率
<code>ql_wifi_p2p_oper_class_channel_get()</code>	获取 Wi-Fi P2P 设备的工作频率
<code>ql_wifi_p2p_ssid_postfix_set()</code>	设置 Wi-Fi P2P 模式的 SSID 后缀名
<code>ql_wifi_p2p_ssid_postfix_get()</code>	获取 Wi-Fi P2P 模式的 SSID 后缀名
<code>ql_wifi_p2p_set_enable_status_ind_cb()</code>	注册 Wi-Fi P2P 模式的启用状态回调函数
<code>ql_wifi_p2p_enable()</code>	启用 Wi-Fi P2P 功能
<code>ql_wifi_p2p_disable()</code>	禁用 Wi-Fi P2P 功能
<code>ql_wifi_p2p_set_dev_found_ind_cb()</code>	注册 P2P 设备扫描信息回调函数
<code>ql_wifi_p2p_dev_find_start()</code>	开始扫描周围的 P2P 设备

<code>ql_wifi_p2p_dev_find_stop()</code>	停止扫描周围的 P2P 设备
<code>ql_wifi_p2p_get_status()</code>	获取 Wi-Fi P2P 模式的连接状态信息
<code>ql_wifi_p2p_set_status_ind_cb()</code>	注册 Wi-Fi P2P 模式连接状态回调函数
<code>ql_wifi_p2p_set_peer_dev_req_ind_cb()</code>	注册对端 P2P 设备连接请求回调函数
<code>ql_wifi_p2p_connect()</code>	连接对端 P2P 设备
<code>ql_wifi_p2p_disconnect()</code>	断开对端 P2P 设备

备注

1. 若无特别说明，不可在任何回调函数中调用上述 Wi-Fi API，否则会对后续消息的处理造成影响。
2. 在调用 `ql_wifi_ap_start()` 或 `ql_wifi_sta_start()` 之前，须调用 `ql_wifi_work_mode_set()` 设置正确的 Wi-Fi 工作模式；若没有预先设置 Wi-Fi 工作模式，则默认的工作模式为单 AP 模式。
3. 在调用 `ql_wifi_ap_start()` 或 `ql_wifi_sta_start()` 之前，须调用 `ql_wifi_enable()` 启用 Wi-Fi。
4. 在调用 `ql_wifi_ap_start()` 或 `ql_wifi_sta_start()` 之前，用户可根据自身需求，调用其它配置接口进行相关的配置。

3.3. 函数详解

3.3.1. ql_wifi_init

该函数用于初始化 Wi-Fi 服务。

● 函数原型

```
int ql_wifi_init(void);
```

● 参数

无

● 返回值

`QL_ERR_OK`

函数执行成功

`QL_ERR_SERVICE_NOT_READY`

函数执行失败，Wi-Fi 服务未准备就绪，建议重试；

其他值

函数执行失败，错误码详见 `ql_type.h`。

备注

使用其它 Wi-Fi API 之前，必须先调用该函数初始化 Wi-Fi 服务。

3.3.2. ql_wifi_deinit

该函数用于去初始化 Wi-Fi 服务。

● 函数原型

```
int ql_wifi_deinit(void);
```

● 参数

无

● 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.3. ql_wifi_enable

该函数用于开启 Wi-Fi 功能。

● 函数原型

```
int ql_wifi_enable (void);
```

● 参数

无

● 返回值

QL_ERR_OK 函数执行成功
QL_ERR_NOT_INIT 函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT 函数执行失败，Wi-Fi 服务出错；
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.4. ql_wifi_disable

该函数用于关闭 Wi-Fi 功能。

- 函数原型

```
int ql_wifi_disable (void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.5. ql_wifi_set_enable_status_ind_cb

该函数用于注册 Wi-Fi 状态回调函数。

- 函数原型

```
int ql_wifi_set_enable_status_ind_cb(ql_wifi_enable_status_ind_cb_f cb);
```

- 参数

cb:

[In] Wi-Fi 状态回调函数。详见第 3.3.5.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.5.1. ql_wifi_enable_status_ind_cb_f

该回调函数用于处理 Wi-Fi 状态。

- 函数原型

```
typedef void (*ql_wifi_enable_status_ind_cb_f)(QL_WIFI_ENABLE_STATUS_E pre_status,
QL_WIFI_ENABLE_STATUS_E status);
```

- 参数

pre_status:

[In] 先前的 Wi-Fi 状态。详见第 3.3.5.2 章。

status:

[In] 当前的 Wi-Fi 状态。详见第 3.3.5.2 章。

- 返回值

无

3.3.5.2. QL_WIFI_ENABLE_STATUS_E

Wi-Fi 状态枚举定义如下：

```
typedef enum QL_WIFI_STATUS_ENUM
{
    QL_WIFI_STATUS_DISABLED = 0,
    QL_WIFI_STATUS_ENABLED,
    QL_WIFI_STATUS_ERROR
} QL_WIFI_ENABLE_STATUS_E;
```

- 成员

成员	描述
<i>QL_WIFI_STATUS_DISABLED</i>	已禁用 Wi-Fi
<i>QL_WIFI_STATUS_ENABLED</i>	已启用 Wi-Fi
<i>QL_WIFI_STATUS_ERROR</i>	启用 Wi-Fi 出错

3.3.6. ql_wifi_work_mode_set

该函数用于设置 Wi-Fi 工作模式。

- 函数原型

```
int ql_wifi_work_mode_set(QL_WIFI_WORK_MODE_E mode);
```

- 参数

mode:

[In] Wi-Fi 工作模式。详见第 3.3.6.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.6.1. QL_WIFI_WORK_MODE_E

Wi-Fi 工作模式枚举定义如下：

```
typedef enum QL_WIFI_WORK_MODE_ENUM
{
    QL_WIFI_WORK_MODE_MIN = -1,
    QL_WIFI_WORK_MODE_STA = 0,
    QL_WIFI_WORK_MODE_AP0,
    QL_WIFI_WORK_MODE_MAX
} QL_WIFI_WORK_MODE_E;
```

- 成员

成员	描述
QL_WIFI_WORK_MODE_STA	STA 模式
QL_WIFI_WORK_MODE_AP0	单 AP 模式

3.3.7. ql_wifi_set_bridge

该函数用于设置当前 Wi-Fi 加入的网桥。

- 函数原型

```
int ql_wifi_set_bridge(char *bridge);
```

- 参数

bridge:

[In] 需加入的网桥名称，若不设置则默认使用网桥 *bridge0*。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.8. ql_wifi_work_mode_get

该函数用于获取当前 Wi-Fi 工作模式。

- 函数原型

```
int ql_wifi_work_mode_get(QL_WIFI_WORK_MODE_E *p_mode);
```

- 参数

p_mode:

[Out] 当前的 Wi-Fi 工作模式。详见第 3.3.6.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.9. ql_wifi_ap_ssid_set

该函数用于设置 AP 模式的 SSID。

- 函数原型

```
int ql_wifi_ap_ssid_set(QL_WIFI_AP_INDEX_E idx, const char *ssid);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

ssid:

[In] SSID。SSID 不可超过 32 字节，且不为空。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.9.1. QL_WIFI_AP_INDEX_E

AP 模式索引枚举定义如下：

```
typedef enum QL_WIFI_INDEX_ENUM
{
    QL_WIFI_AP_INDEX_MIN = -1,
    QL_WIFI_AP_INDEX_AP0 = 0,
    QL_WIFI_AP_INDEX_MAX
} QL_WIFI_AP_INDEX_E;
```

- 成员

成员	描述
QL_WIFI_AP_INDEX_AP0	热点 0 的索引

3.3.10. ql_wifi_ap_ssid_get

该函数用于获取 AP 模式的 SSID。

- 函数原型

```
int ql_wifi_ap_ssid_get(QL_WIFI_AP_INDEX_E idx, char *ssid_buf, int buf_len);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

ssid_buf:

[Out] 指针，指向存放当前 SSID 的缓存。

buf_len:

[In] 缓存大小。缓存大小建议设置为 33 字节。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.11. ql_wifi_ap_ssid_hidden_set

该函数用于设置 AP 模式的 SSID 隐藏状态。

- 函数原型

```
int ql_wifi_ap_ssid_hidden_set(QL_WIFI_AP_INDEX_E idx, int ssid_hidden);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

ssid_hidden:

[In] SSID 隐藏状态。

0	不隐藏 SSID
1	隐藏 SSID

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.12. ql_wifi_ap_ssid_hidden_get

该函数用于获取 AP 模式的 SSID 隐藏状态。

- 函数原型

```
int ql_wifi_ap_ssid_hidden_get(QL_WIFI_AP_INDEX_E idx, int *p_ssid_hidden);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_ssid_hidden:

[Out] AP 模式的 SSID 隐藏状态。

0	不隐藏 SSID
1	隐藏 SSID

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.13. ql_wifi_ap_mode_set

该函数用于设置 AP 模式的工作协议模式。

- 函数原型

```
int ql_wifi_ap_mode_set(QL_WIFI_AP_INDEX_E idx, QL_WIFI_AP_MODE_TYPE_E mode);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

mode:

[In] 工作协议模式。详见第 3.3.13.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.13.1. QL_WIFI_AP_MODE_TYPE_E

AP 模式的工作协议模式枚举定义如下：

```
typedef enum QL_WIFI_AP_MODE_TYPE_ENUM
{
    QL_WIFI_AP_MODE_MIN = -1,
    QL_WIFI_AP_MODE_80211B = 0,
    QL_WIFI_AP_MODE_80211BG,
    QL_WIFI_AP_MODE_80211BGN,
    QL_WIFI_AP_MODE_80211AX_2G,
    QL_WIFI_AP_MODE_MAX
} QL_WIFI_AP_MODE_TYPE_E;
```

- 成员

成员	描述
<i>QL_WIFI_AP_MODE_80211B</i>	IEEE 802.11b (2.4 GHz)
<i>QL_WIFI_AP_MODE_80211BG</i>	IEEE 802.11b/g (2.4 GHz)
<i>QL_WIFI_AP_MODE_80211BGN</i>	IEEE 802.11b/g/n (2.4 GHz)
<i>QL_WIFI_AP_MODE_80211AX_2G</i>	IEEE 802.11ax (2.4 GHz)

3.3.14. ql_wifi_ap_mode_get

该函数用于获取 AP 模式的工作协议模式。

- 函数原型

```
int ql_wifi_ap_mode_get(QL_WIFI_AP_INDEX_E idx, QL_WIFI_AP_MODE_TYPE_E *p_mode);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_mode:

[Out] 当前的协议模式。详见第 3.3.13.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.15. ql_wifi_ap_bandwidth_set

该函数用于设置 AP 模式的带宽。

- 函数原型

```
int ql_wifi_ap_bandwidth_set(QL_WIFI_AP_INDEX_E idx,
                             QL_WIFI_BANDWIDTH_E bandwidth);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

bandwidth:

[In] AP 模式的带宽。详见第 3.3.15.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.15.1. QL_WIFI_BANDWIDTH_E

AP 模式的带宽类型枚举定义如下：

```
typedef enum QL_WIFI_BANDWIDTH_ENUM
{
    QL_WIFI_BANDWIDTH_MIN = -1,
    QL_WIFI_BANDWIDTH_20MHZ = 0,
    QL_WIFI_BANDWIDTH_40MHZ,
    QL_WIFI_BANDWIDTH_80MHZ,
    QL_WIFI_BANDWIDTH_160MHZ,
```

```
QL_WIFI_BANDWIDTH_MAX
} QL_WIFI_BANDWIDTH_E;
```

● 成员

成员	描述
<code>QL_WIFI_BANDWIDTH_20MHZ</code>	20 MHz
<code>QL_WIFI_BANDWIDTH_40MHZ</code>	40 MHz
<code>QL_WIFI_BANDWIDTH_80MHZ</code>	80 MHz
<code>QL_WIFI_BANDWIDTH_160MHZ</code>	160 MHz

3.3.16. ql_wifi_ap_bandwidth_get

该函数用于获取 AP 模式的带宽。

● 函数原型

```
int ql_wifi_ap_bandwidth_get(QL_WIFI_AP_INDEX_E idx,
                             QL_WIFI_BANDWIDTH_E *p_bandwidth);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_bandwidth:

[Out] 当前的 AP 模式的带宽。详见第 3.3.15.1 章。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.17. ql_wifi_ap_channel_set

该函数用于设置 AP 模式的信道。

● 函数原型

```
int ql_wifi_ap_channel_set(QL_WIFI_AP_INDEX_E idx, int channel);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

channel:

[In] 信道，设置 2.4 GHz 的信道：

0	自适应
1/2/3/4/5/6/7/8/9/10/11/12/13/14	2.4 GHz 信道

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.18. ql_wifi_ap_channel_get

该函数用于获取 AP 模式的信道。

● 函数原型

```
int ql_wifi_ap_channel_get(QL_WIFI_AP_INDEX_E idx, int *p_channel);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_channel:

[Out] AP 当前使用的信道。

0	自适应
1/2/3/4/5/6/7/8/9/10/11/12/13/14	2.4 GHz 信道

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；

QL_ERR_INVALID_ARG 函数执行失败，无效参数；
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.19. ql_wifi_ap_max_sta_num_set

该函数用于设置 AP 模式终端最大连接数量。

● 函数原型

```
int ql_wifi_ap_max_sta_num_set(QL_WIFI_AP_INDEX_E idx, int max_sta_num);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

max_sta_num:

[In] AP 模式终端最大连接数量，取值范围：1~32。

● 返回值

QL_ERR_OK 函数执行成功
QL_ERR_NOT_INIT 函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT 函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG 函数执行失败，无效参数；
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.20. ql_wifi_ap_max_sta_num_get

该函数用于获取 AP 模式终端最大连接数量。

● 函数原型

```
int ql_wifi_ap_max_sta_num_get(QL_WIFI_AP_INDEX_E idx, int *p_max_sta_num);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_max_sta_num:

[Out] 当前 AP 模式终端最大连接数量。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.21. ql_wifi_ap_country_code_set

该函数用于设置 AP 模式的国家码。

- 函数原型

```
int ql_wifi_ap_country_code_set(QL_WIFI_AP_INDEX_E idx, const char *country_code);
```

- 参数

`idx`:

[In] AP 模式索引。详见第 3.3.9.1 章。

`country_code`:

[In] 国家码。默认值：CN（表示中国）；最大长度：2 个字节。不可为空。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.22. ql_wifi_ap_country_code_get

该函数用于获取 AP 模式的国家码。

- 函数原型

```
int ql_wifi_ap_country_code_get(QL_WIFI_AP_INDEX_E idx,
                                char *country_code_buf,
                                int buf_len);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

country_code_buf:

[Out] 指针，指向存放当前国家码的缓存。

buf_len:

[In] 缓存大小。缓存大小建议设置为 3 字节。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.23. ql_wifi_ap_isolate_set

该函数用于设置 AP 隔离状态。

- 函数原型

```
int ql_wifi_ap_isolate_set(QL_WIFI_AP_INDEX_E idx, int isolate);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

isolate:

[In] AP 隔离状态。

0	AP 不隔离
1	AP 隔离

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.24. ql_wifi_ap_isolate_get

该函数用于获取 AP 隔离状态。

● 函数原型

```
int ql_wifi_ap_isolate_get(QL_WIFI_AP_INDEX_E idx, int *p_isolate);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_isolate:

[Out] AP 当前的隔离状态。

0	AP 不隔离
1	AP 隔离

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.25. ql_wifi_ap_mac_acl_rule_set

该函数用于设置 AP 模式的 MAC 地址访问规则。

● 函数原型

```
int ql_wifi_ap_mac_acl_rule_set(QL_WIFI_AP_INDEX_E idx,
                                QL_WIFI_MAC_ACL_RULE_E acl_rule);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

acl_rule:

[In] MAC 地址访问规则。详见第 3.3.25.1 章。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.25.1. QL_WIFI_MAC_ACL_RULE_E

MAC 地址的访问规则枚举定义如下：

```
typedef enum QL_WIFI_MAC_RULE_ENUM
{
    QL_WIFI_MAC_ACL_RULE_MIN = -1,
    QL_WIFI_MAC_ACL_RULE_NONE = 0,
    QL_WIFI_MAC_ACL_RULE_BLACK,
    QL_WIFI_MAC_ACL_RULE_WHITE,
    QL_WIFI_MAC_ACL_RULE_MAX
} QL_WIFI_MAC_ACL_RULE_E;
```

● 成员

成员	描述
QL_WIFI_MAC_ACL_RULE_NONE	无规则
QL_WIFI_MAC_ACL_RULE_BLACK	黑名单
QL_WIFI_MAC_ACL_RULE_WHITE	白名单

3.3.26. ql_wifi_ap_mac_acl_rule_get

该函数用于获取 AP 模式的 MAC 地址访问规则。

● 函数原型

```
int ql_wifi_ap_mac_acl_rule_get(QL_WIFI_AP_INDEX_E idx,
                                QL_WIFI_MAC_ACL_RULE_E *p_acl_rule);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_acl_rule:

[Out] 当前的 MAC 地址访问规则。详见第 3.3.25.1 章。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化 Wi-Fi 服务；
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，Wi-Fi 服务出错；
<i>QL_ERR_INVALID_ARG</i>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.27. ql_wifi_ap_acl_mac_set

该函数用于为当前 MAC 地址访问规则添加或者删除指定的 MAC 地址。

- 函数原型

```
int ql_wifi_ap_acl_mac_set(QL_WIFI_AP_INDEX_E idx,
                           QL_WIFI_ACL_MAC_CMD_E cmd,
                           const char *macaddr);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

cmd:

[In] MAC 地址操作类型。详见第 3.3.27.1 章。

macaddr:

[In] MAC 地址。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化 Wi-Fi 服务；
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，Wi-Fi 服务出错；
<i>QL_ERR_INVALID_ARG</i>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.27.1. QL_WIFI_ACL_MAC_CMD_E

MAC 地址访问规则中对 MAC 地址的操作类型枚举定义如下：

```
typedef enum
{
    QL_WIFI_ACL_MAC_CMD_DEL = 0,
    QL_WIFI_ACL_MAC_CMD_ADD
}QL_WIFI_ACL_MAC_CMD_E;
```

● 成员

成员	描述
<code>QL_WIFI_ACL_MAC_CMD_DEL</code>	删除 MAC 地址
<code>QL_WIFI_ACL_MAC_CMD_ADD</code>	添加 MAC 地址

3.3.28. ql_wifi_ap_acl_mac_get

该函数用于获取当前 MAC 地址访问规则中所有的 MAC 地址。

● 函数原型

```
int ql_wifi_ap_acl_mac_get(QL_WIFI_AP_INDEX_E idx, ql_wifi_acl_mac_list_t *p_mac_list);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_mac_list:

[Out] MAC 地址列表。详见第 3.3.28.1 章。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.28.1. ql_wifi_acl_mac_list_t

MAC 地址访问规则中的 MAC 地址列表结构体定义如下：

```
typedef struct
{
```

```
int cnt;
ql_wifi_mac_addr_t addr[QL_WIFI_MAX_ACL_MAC_CNT];
}ql_wifi_acl_mac_list_t;
```

- 参数

类型	参数	描述
int	<i>cnt</i>	MAC 地址个数，最大不超过 32。
<i>ql_wifi_mac_addr_t</i>	<i>addr</i>	MAC 地址，详见第 3.3.28.2 章。 <i>QL_WIFI_MAX_ACL_MAC_CNT</i> 定义在接口头文件 <i>ql_wifi_common.h</i> 中，值为 32。

3.3.28.2. ql_wifi_mac_addr_t

MAC 地址访问规则中的 MAC 地址定义结构体定义如下：

```
typedef struct
{
    char macaddr[18];
}ql_wifi_mac_addr_t;
```

- 参数

类型	参数	描述
char	<i>macaddr</i>	MAC 地址 格式为：%02X:%02X:%02X:%02X:%02X:%02X 例如：2F:3F:4F:5F:6F:7F

3.3.29. ql_wifi_ap_auth_set

该函数用于设置 AP 模式的安全认证。

- 函数原型

```
int ql_wifi_ap_auth_set(QL_WIFI_AP_INDEX_E idx, ql_wifi_ap_auth_t *p_auth);
```

- 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_auth:

[In] 安全认证信息。详见第3.3.29.1章。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.29.1. ql_wifi_ap_auth_t

AP 模式的安全认证结构体定义如下：

```
typedef struct
{
    QL_WIFI_AUTH_E auth;
    union
    {
        struct
        {
            int default_index;
            char passwd[4][64];
        } wep;

        struct
        {
            QL_WIFI_AUTH_WPA_PSK_E pairwise;
            char passwd[64];
            int group_rekey;
        } wpa_psk;
    } info;
} ql_wifi_ap_auth_t;
```

● 参数

类型	参数	描述
QL_WIFI_AUTH_E	<i>auth</i>	安全认证类型；详见第3.3.29.2章。
int	<i>default_index</i>	WEP 的配置索引
char	<i>passwd</i>	WEP 的密码

QL_WIFI_AUTH_WPA_PSK_E	pairwise	加密方式：详见第3.3.29.3章。
char	passwd	WPA_PSK 的密码
int	group_rekey	组密钥更新周期

3.3.29.2. QL_WIFI_AUTH_E

安全认证类型枚举定义如下：

```
typedef enum QL_WIFI_AUTH_ENUM
{
    QL_WIFI_AUTH_MIN = -1,
    QL_WIFI_AUTH_OPEN = 0,
    QL_WIFI_AUTH_WEP,
    QL_WIFI_AUTH_WPA_PSK,
    QL_WIFI_AUTH_WPA2_PSK,
    QL_WIFI_AUTH_WPA3_PSK,
    QL_WIFI_AUTH_WPA_WPA2_PSK_BOTH,
    QL_WIFI_AUTH_WPA2_WPA3_PSK_BOTH,
    QL_WIFI_AUTH_WPA_WPA2_WPA3_PSK_ALL,
    QL_WIFI_AUTH_MAX
} QL_WIFI_AUTH_E;
```

● 成员

成员	描述
QL_WIFI_AUTH_OPEN	OPEN
QL_WIFI_AUTH_WEP	WEP
QL_WIFI_AUTH_WPA_PSK	WPA 个人版
QL_WIFI_AUTH_WPA2_PSK	WPA2 个人版
QL_WIFI_AUTH_WPA3_PSK	WPA3 个人版
QL_WIFI_AUTH_WPA_WPA2_PSK_BOTH	WPA&WPA2 个人版
QL_WIFI_AUTH_WPA2_WPA3_PSK_BOTH	WPA2&WPA3 个人版
QL_WIFI_AUTH_WPA_WPA2_WPA3_PSK_ALL	WPA&WPA2&WPA3 个人版

3.3.29.3. QL_WIFI_AUTH_WPA_PSK_E

加密方式枚举定义如下：

```
typedef enum QL_WIFI_AUTH_WPA_PSK_ENUM
{
    QL_WIFI_AUTH_WPA_PAIRWISE_MIN = -1,
    QL_WIFI_AUTH_WPA_PAIRWISE_AUTO = 0,
    QL_WIFI_AUTH_WPA_PAIRWISE_TKIP,
    QL_WIFI_AUTH_WPA_PAIRWISE_AES,
    QL_WIFI_AUTH_WPA_PAIRWISE_MAX
} QL_WIFI_AUTH_WPA_PSK_E;
```

● 成员

成员	描述
QL_WIFI_AUTH_WPA_PAIRWISE_AUTO	自动模式（包含 TKIP 和 AES）
QL_WIFI_AUTH_WPA_PAIRWISE_TKIP	TKIP
QL_WIFI_AUTH_WPA_PAIRWISE_AES	AES

备注

要使 Wi-Fi 设备连接时达到最大兼容性，建议在 AP 工作模式下配置加密方式为 AES；在 STA 工作模式下配置加密模式为自动模式。

3.3.30. ql_wifi_ap_auth_get

该函数用于获取 AP 模式的安全认证信息。

● 函数原型

```
int ql_wifi_ap_auth_get(QL_WIFI_AP_INDEX_E idx, ql_wifi_ap_auth_t *p_auth);
```

● 参数

idx:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_auth:

[Out] 安全认证信息。详见第 3.3.29.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.31. ql_wifi_ap_set_status_ind_cb

该函数用于注册 AP 模式状态回调函数。

- 函数原型

```
int ql_wifi_ap_set_status_ind_cb(ql_wifi_ap_status_ind_cb_f cb);
```

- 参数

cb:

[In] AP 模式状态回调函数。详见第 3.3.31.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.31.1. ql_wifi_ap_status_ind_cb_f

该回调函数用于处理 AP 模式状态。

- 函数原型

```
typedef void (*ql_wifi_ap_status_ind_cb_f)(QL_WIFI_AP_INDEX_E index,
                                           QL_WIFI_AP_STATUS_E pre_status,
                                           ql_wifi_ap_status_t *p_msg);
```

- 参数

index:

[In] AP 模式索引。详见第 3.3.9.1 章。

pre_status:

[In] 先前的 AP 模式状态。详见第 3.3.31.2 章。

p_msg:

[In] 当前的 AP 模式状态等信息。详见第 3.3.31.3 章。

- 返回值

无

3.3.31.2. QL_WIFI_AP_STATUS_E

AP 模式的状态枚举定义如下：

```
typedef enum QL_WIFI_AP_STATUS_ENUM
{
    QL_WIFI_AP_STATUS_NONE,
    QL_WIFI_AP_STATUS_IDLE,
    QL_WIFI_AP_STATUS_ENABLING,
    QL_WIFI_AP_STATUS_ENABLED,
    QL_WIFI_AP_STATUS_DISABLING,
    QL_WIFI_AP_STATUS_ERROR
} QL_WIFI_AP_STATUS_E;
```

- 成员

成员	描述
<i>QL_WIFI_AP_STATUS_NONE</i>	未设置相应的 Wi-Fi 工作模式
<i>QL_WIFI_AP_STATUS_IDLE</i>	设置了相应的 Wi-Fi 工作模式，但未开启 AP 功能。
<i>QL_WIFI_AP_STATUS_ENABLING</i>	AP 模式启用中（暂不支持）
<i>QL_WIFI_AP_STATUS_ENABLED</i>	AP 模式已启用
<i>QL_WIFI_AP_STATUS_DISABLING</i>	AP 模式禁用中（暂不支持）
<i>QL_WIFI_AP_STATUS_ERROR</i>	出错状态

3.3.31.3. ql_wifi_ap_status_t

AP 模式的状态信息结构体定义如下：

```
typedef struct ql_wifi_ind_ap_status_struct
{
    QL_WIFI_AP_STATUS_E status;
    char ifname[32];
    char bssid[18];
}ql_wifi_ap_status_t;
```

● 参数

类型	参数	描述
QL_WIFI_AP_STATUS_E	status	AP 模式的状态；详见第 3.3.31.2 章。
char	ifname	接口名，例如 wlan0、wlan1 等。
char	bssid	BSSID，基本服务集标识符。

3.3.32. ql_wifi_ap_start

该函数用于开启 AP 功能。

● 函数原型

```
int ql_wifi_ap_start(QL_WIFI_AP_INDEX_E index);
```

● 参数

index:

[In] AP 模式索引。详见第 3.3.9.1 章。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 ql_type.h。

3.3.33. ql_wifi_ap_stop

该函数用于关闭 AP 功能。

- 函数原型

```
int ql_wifi_ap_stop(QL_WIFI_AP_INDEX_E index);
```

- 参数

index:

[In] AP 模式索引。详见第 3.3.9.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.34. ql_wifi_ap_get_status

该函数用于获取 AP 模式的状态信息。

- 函数原型

```
int ql_wifi_ap_get_status(QL_WIFI_AP_INDEX_E index, ql_wifi_ap_status_t *p_sta);
```

- 参数

index:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_sta:

[Out] AP 模式的状态信息。详见第 3.3.31.3 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.35. ql_wifi_sta_ssid_set

该函数用于设置 STA 模式的 SSID。

- 函数原型

```
int ql_wifi_sta_ssid_set(const char *ssid);
```

- 参数

ssid:

[In] SSID。SSID 不可超过 32 字节，且不为空。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.36. ql_wifi_sta_ssid_get

该函数用于获取 STA 模式的 SSID。

- 函数原型

```
int ql_wifi_sta_ssid_get(char *ssid_buf, int buf_len);
```

- 参数

ssid_buf:

[Out] 指针，指向存放当前 SSID 的缓存。

buf_len:

[In] 缓存大小。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.37. ql_wifi_sta_auth_set

该函数用于设置 STA 模式的安全认证。

- 函数原型

```
int ql_wifi_sta_auth_set(ql_wifi_sta_auth_t *p_auth);
```

- 参数

p_auth:

[In] 安全认证信息。详见第 3.3.37.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.37.1. ql_wifi_sta_auth_t

STA 模式的安全认证结构体定义如下：

```
typedef struct
{
    QL_WIFI_AUTH_E auth;
    union
    {
        struct
        {
            char passwd[64];
        } wep;

        struct
        {
            QL_WIFI_AUTH_WPA_PSK_E pairwise;
            char passwd[64];
        } wpa_psk;
    } info;
} ql_wifi_sta_auth_t;
```

● 参数

类型	参数	描述
QL_WIFI_AUTH_E	auth	安全认证类型。详见第 3.3.29.2 章。
char	passwd	WEP 的密码。
QL_WIFI_AUTH_WPA_PSK_E	pairwise	加密方式。详见第 3.3.29.3 章。
char	passwd	WPA_PSK 的密码。

3.3.38. ql_wifi_sta_auth_get

该函数用于获取 STA 模式的安全认证信息。

● 函数原型

```
int ql_wifi_sta_auth_get(ql_wifi_sta_auth_t *p_auth);
```

● 参数

p_auth:

[Out] 当前的安全认证信息。详见第 3.3.37.1 章。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 ql_type.h。

3.3.39. ql_wifi_sta_set_status_ind_cb

该函数用于注册 STA 模式状态回调函数。

● 函数原型

```
int ql_wifi_sta_set_status_ind_cb(ql_wifi_sta_status_ind_cb_f cb);
```

● 参数

cb:

[In] STA 模式状态回调函数。详见第 3.3.39.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.39.1. ql_wifi_sta_status_ind_cb_f

该回调函数用于处理 STA 模式状态。

- 函数原型

```
typedef void (*ql_wifi_sta_status_ind_cb_f)(QL_WIFI_STA_STATUS_E pre_status,
                                             ql_wifi_sta_status_t *p_msg);
```

- 参数

pre_status:

[In] 之前的 STA 模式状态。详见第 3.3.39.2 章。

p_msg:

[In] 当前的 STA 模式状态等信息。详见第 3.3.39.3 章。

- 返回值

无

3.3.39.2. QL_WIFI_STA_STATUS_E

STA 模式的状态枚举定义如下：

```
typedef enum QL_WIFI_STA_STATUS_ENUM
{
    QL_WIFI_STA_STATUS_NONE,
    QL_WIFI_STA_STATUS_IDLE,
    QL_WIFI_STA_STATUS_CONNECTING,
    QL_WIFI_STA_STATUS_ASSOCIATED,
    QL_WIFI_STA_STATUS_CONNECTED,
    QL_WIFI_STA_STATUS_DISCONNECTED,
    QL_WIFI_STA_STATUS_ERROR
} QL_WIFI_STA_STATUS_E;
```

● 成员

成员	描述
<code>QL_WIFI_STA_STATUS_NONE</code>	未设置相应的 Wi-Fi 工作模式
<code>QL_WIFI_STA_STATUS_IDLE</code>	设置了相应的 Wi-Fi 工作模式，但未开启 STA 功能
<code>QL_WIFI_STA_STATUS_CONNECTING</code>	正在连接 AP
<code>QL_WIFI_STA_STATUS_ASSOCIATED</code>	已经连上 AP，但未分配 IP 地址
<code>QL_WIFI_STA_STATUS_CONNECTED</code>	已经连上 AP，并且已分配 IP 地址
<code>QL_WIFI_STA_STATUS_DISCONNECTED</code>	和 AP 断开连接
<code>QL_WIFI_STA_STATUS_ERROR</code>	出错状态

3.3.39.3. ql_wifi_sta_status_t

STA 模式的状态信息结构体定义如下：

```
typedef struct ql_wifi_sta_status_struct
{
    QL_WIFI_STA_STATUS_E status;
    char ifname[32];
    char ap_bssid[18];
    int rssi;
    uint8_t has_addr;
    ql_net_addr_t addr;
    uint8_t has_addr6;
    ql_net_addr6_t addr6;
    QL_WIFI_REASON_CODE_E reason_code;
    QL_WIFI_SUB_REASON_CODE_E sub_reason_code;
} ql_wifi_sta_status_t;
```

● 参数

类型	参数	描述
<code>QL_WIFI_STA_STATUS_E</code>	<code>status</code>	STA 模式状态。详见第 3.3.39.2 章。
<code>char</code>	<code>ifname</code>	网络接口名
<code>char</code>	<code>ap_bssid</code>	BSSID，基本服务集标识符。
<code>int</code>	<code>rssi</code>	STA 设备连接热点时的信号强度；单

		位：dBm；-9999 表示无效值。
uint8_t	has_addr	是否为 IPv4 地址。 1 是 0 否
ql_net_addr_t	addr	IPv4 地址，ql_net_addr_t 定义在接口头文件 ql_net_common.h 中。
uint8_t	has_addr6	是否为 IPv6 地址。 1 是 0 否
ql_net_addr6_t	addr6	IPv6 地址，ql_net_addr6_t 定义在接口头文件 ql_net_common.h 中。
QL_WIFI_REASON_CODE_E	reason_code	STA 错误原因码，QL_WIFI_REASON_CODE_E 定义在接口头文件 ql_wifi_common.h 中。
QL_WIFI_SUB_REASON_CODE_E	sub_reason_code	STA 错误次要原因码，QL_WIFI_SUB_REASON_CODE_E 定义在接口头文件 ql_wifi_common.h 中。

3.3.40. ql_wifi_sta_start

该函数用于开启 STA 功能。

● 函数原型

```
int ql_wifi_sta_start(void);
```

● 参数

无

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 ql_type.h。

备注

调用该函数前，须开启 Wi-Fi 功能，且将 Wi-Fi 工作模式设置为 STA 模式。

3.3.41. ql_wifi_sta_stop

该函数用于关闭 STA 功能。

- 函数原型

```
int ql_wifi_sta_stop(void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.42. ql_wifi_sta_get_status

该函数用于获取 STA 模式的状态信息。

- 函数原型

```
int ql_wifi_sta_get_status(ql_wifi_sta_status_t *p_sta);
```

- 参数

p_sta:

[Out] STA 模式的状态信息。详见第 3.3.39.3 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.43. ql_wifi_sta_start_scan

该函数用于 STA 模式扫描外部热点。

- 函数原型

```
int ql_wifi_sta_start_scan(void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.44. ql_wifi_sta_set_scan_result_ind_cb

该函数用于注册扫描结果回调函数。

- 函数原型

```
int ql_wifi_sta_set_scan_result_ind_cb(ql_wifi_sta_scan_result_ind_cb_f cb);
```

- 参数

cb:

[In] 扫描结果回调函数。详见第 3.3.44.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.44.1. ql_wifi_sta_scan_result_ind_cb_f

该回调函数用于处理 STA 模式下的扫描结果。

- 函数原型

```
typedef void (*ql_wifi_sta_scan_result_ind_cb_f)(ql_wifi_sta_scan_result_t *p_msg);
```

- 参数

p_msg:

[In] 外部热点信息扫描结果。详见第 3.3.44.2 章。

- 返回值

无

3.3.44.2. ql_wifi_sta_scan_result_t

STA 模式外部热点信息扫描结果结构体定义如下：

```
typedef struct ql_wifi_sta_scan_result_struct
{
    QL_WIFI_SCAN_REASON_CODE_E reason_code;
    ql_wifi_sta_scan_list_t scan_list;
}ql_wifi_sta_scan_result_t;
```

- 参数

类型	参数	描述
QL_WIFI_SCAN_REASON_CODE_E	<i>reason_code</i>	扫描结果码；详见第 3.3.43.5 章。
<i>ql_wifi_sta_scan_list_t</i>	<i>scan_list</i>	热点信息扫描列表；详见第 3.3.44.3 章。 当 <i>reason_code</i> 为 QL_WIFI_SCAN_REASON_CODE_SUCCESS 时该字段有效。

3.3.44.3. ql_wifi_sta_scan_list_t

STA 模式扫描的外部热点信息列表，结构体定义如下：

```
typedef struct ql_wifi_sta_scan_list_struct
{
```

```
int cnt;
ql_wifi_sta_scan_info_t info[QL_WIFI_MAX_SCAN_INFO_CNT];
}ql_wifi_sta_scan_list_t;
```

● 参数

类型	参数	描述
int	<i>cnt</i>	扫描到的外部热点个数。
<i>ql_wifi_sta_scan_info_t</i>	<i>info</i>	扫描的外部热点信息。详见第3.3.44.4章。 <i>QL_WIFI_MAX_SCAN_INFO_CNT</i> 定义在接口头文件 <i>ql_wifi_common.h</i> 中，值为 60。

3.3.44.4. ql_wifi_sta_scan_info_t

STA 模式扫描的外部热点信息结构体定义如下：

```
typedef struct ql_wifi_sta_scan_info_struct
{
    char bssid[18];
    char essid[33];
    int signal;
    int frequency;
    QL_WIFI_AUTH_E auth;
}ql_wifi_sta_scan_info_t;
```

● 参数

类型	参数	描述
char	<i>bssid</i>	BSSID，基本服务集标识符。
char	<i>essid</i>	扩展服务集标识符。
int	<i>signal</i>	信号强度。单位：dBm。
int	<i>frequency</i>	当前工作频率。单位：Hz。
<i>QL_WIFI_AUTH_E</i>	<i>auth</i>	认证方式；详见第3.3.29.2章。

3.3.44.5. QL_WIFI_SCAN_REASON_CODE_E

扫描结果码枚举定义如下：

```
typedef enum QL_WIFI_SCAN_REASON_CODE
{
    QL_WIFI_SCAN_REASON_CODE_SUCCESS = 0,
    QL_WIFI_SCAN_REASON_CODE_BUSY = 1,
    QL_WIFI_SCAN_REASON_CODE_BREAK_START = 2,
    QL_WIFI_SCAN_REASON_CODE_BREAK_STOP = 3,
    QL_WIFI_SCAN_REASON_CODE_UNKNOW = 4
} QL_WIFI_SCAN_REASON_CODE_E;
```

● 成员

成员	描述
<code>QL_WIFI_SCAN_REASON_CODE_SUCCESS</code>	扫描成功
<code>QL_WIFI_SCAN_REASON_CODE_BUSY</code>	扫描繁忙，建议重试。
<code>QL_WIFI_SCAN_REASON_CODE_BREAK_START</code>	扫描被 <code>ql_wifi_sta_start()</code> 打断
<code>QL_WIFI_SCAN_REASON_CODE_BREAK_STOP</code>	扫描被 <code>ql_wifi_sta_stop()</code> 打断
<code>QL_WIFI_SCAN_REASON_CODE_UNKNOW</code>	扫描失败，原因未知。

3.3.45. ql_wifi_set_service_error_cb

该函数用于注册 Wi-Fi 服务异常回调函数。

● 函数原型

```
int ql_wifi_set_service_error_cb(ql_wifi_service_error_cb_f cb);
```

● 参数

cb:

[In] Wi-Fi 服务异常回调函数。只有当 Wi-Fi 服务异常退出才会执行回调函数；详见第 3.3.45.1 章。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.45.1. ql_wifi_service_error_cb_f

该回调函数用于处理 Wi-Fi 服务异常。

- 函数原型

```
typedef void (*ql_wifi_service_error_cb_f)(int error);
```

- 参数

error:

[In] 错误码。

- 返回值

无

备注

Wi-Fi API 使用 C/S (Client/Server) 架构。用户在调用 *ql_wifi_init()* 时，会创建与服务程序 (*ql_wifid*) 的通信连接；在调用其它的 Wi-Fi API 时，则会发送请求给服务程序，服务程序处理完请求后返回处理结果给 Wi-Fi API。若服务程序出现异常退出，用户在调用 Wi-Fi API 时会返回 *QL_ERR_SERVICE_ABORT* 错误码，详见 *ql_type.h*；用户也可调用 *ql_wifi_set_service_error_cb()* 注册服务程序异常的回调函数，用于监控服务程序的异常退出。在检测到服务程序异常退出后，用户的客户端程序可以调用 *ql_wifi_deinit()* 注销 Wi-Fi 服务，再调用 *ql_wifi_init()* 创建和服务程序的通信连接，恢复 Wi-Fi 服务。

3.3.46. ql_wifi_set_ap_sta_connect_ind_cb

该函数用于注册 AP 与 STA 设备连接状态回调函数。

- 函数原型

```
int ql_wifi_set_ap_sta_connect_ind_cb(ql_wifi_ap_sta_connect_ind_cb_f cb);
```

- 参数

cb:

[In] AP 模式和 STA 设备连接状态回调函数。详见第 3.3.46.1 章。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_NOT_INIT

函数执行失败，未初始化 Wi-Fi 服务；

QL_ERR_SERVICE_ABORT 函数执行失败，Wi-Fi 服务出错；
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.46.1. ql_wifi_ap_sta_connect_ind_cb_f

该回调函数用于处理 AP 模式和 STA 设备连接状态。

● 函数原型

```
typedef void (*ql_wifi_ap_sta_connect_ind_cb_f)(QL_WIFI_AP_INDEX_E index,
                                                ql_wifi_sta_connect_status_t *p_msg);
```

● 参数

index:

[In] AP 模式索引。详见第 3.3.9.1 章。

p_msg:

[In] AP 模式和 STA 设备的连接状态。详见第 3.3.46.2 章。

● 返回值

无

3.3.46.2. ql_wifi_sta_connect_status_t

AP 模式和 STA 设备连接状态结构体定义如下：

```
typedef struct
{
    int is_connected;
    char macaddr[18];
}ql_wifi_sta_connect_status_t;
```

● 参数

类型	参数	描述
int	<i>is_connected</i>	连接状态。 1 已连接 0 已断开
char	<i>macaddr</i>	STA 设备的 MAC 地址

3.3.47. ql_wifi_country_code_set

该函数用于设置 Wi-Fi 国家码。

- 函数原型

```
int ql_wifi_country_code_set(const char *country_code);
```

- 参数

country_code:

[In] 国家码。默认值：CN（表示中国）；最大长度：2 个字节。不可为空。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

Wi-Fi 国家码默认为 CN，如需修改国家码，则需在调用 *ql_wifi_enable()* 开启 Wi-Fi 功能之前修改。

3.3.48. ql_wifi_country_code_get

该函数用于获取 Wi-Fi 国家码。

- 函数原型

```
int ql_wifi_country_code_get(char *country_code_buf, int buf_len);
```

- 参数

country_code_buf:

[Out] 指针，指向存储 Wi-Fi 国家码配置的缓存。

buf_len:

[In] 缓存大小。单位：字节。该缓存建议设置大小为 3 字节。

- 返回值

QL_ERR_OK	函数执行成功
-----------	--------

QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.49. ql_wifi_p2p_dev_name_set

该函数用于设置 Wi-Fi P2P 设备名称。

● 函数原型

```
int ql_wifi_p2p_dev_name_set(const char *dev_name);
```

● 参数

dev_name:

[In] P2P 设备名称。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.50. ql_wifi_p2p_dev_name_get

该函数用于获取 Wi-Fi P2P 设备名称。

● 函数原型

```
int ql_wifi_p2p_dev_name_get(char *dev_name_buf, int buf_len);
```

● 参数

dev_name_buf:

[Out] 指针，指向存储 P2P 设备名称配置的缓存。

buf_len:

[In] 缓存大小。单位：字节。

● 返回值

QL_ERR_OK	函数执行成功
-----------	--------

QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.51. ql_wifi_p2p_dev_type_set

该函数用于设置 Wi-Fi P2P 设备类型。

● 函数原型

```
int ql_wifi_p2p_dev_type_set(const char *dev_type);
```

● 参数

dev_type:
[In] P2P 设备类型。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

主要设备类型

格式: <categ>-<OUI>-<subcateg>

<categ> 类别为整数值

<OUI> 组织唯一标识符。四个八位字节的 16 进制编码数值；WPS OUI 默认为 0050F204。

<subcateg> OUI 特定的整型类型数值子类别

举例：

1-0050F204-1（电脑/个人电脑）

1-0050F204-2（计算机/服务器）

5-0050F204-1（存储/NAS）

6-0050F204-1（网络基础设施/AP）

3.3.52. ql_wifi_p2p_dev_type_get

该函数用于获取 Wi-Fi P2P 设备类型。

- 函数原型

```
int ql_wifi_p2p_dev_type_get(char *dev_type_buf, int buf_len);
```

- 参数

dev_type_buf:

[Out] 指针，指向存储 P2P 设备类型配置的缓存。

buf_len:

[In] 缓存大小。单位：字节。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.53. ql_wifi_p2p_oper_class_channel_set

该函数用于设置 Wi-Fi P2P 设备的工作频率。

- 函数原型

```
int ql_wifi_p2p_oper_class_channel_set(int oper_class, int oper_channel);
```

- 参数

oper_class:

[In] 待设置的 operating class。

oper_channel:

[In] 待设置的 operating channel。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

`oper_class` 和 `oper_channel` 共同限定了 P2P 设备的工作频率，暂支持如下取值：

<code>oper_class</code>	<code>oper_channel</code>	<code>frequency</code>	<code>channel spacing</code>
81	1-13	$2407 + 5 * \text{oper_channel}$	25MHz
82	14	$2414 + 5 * \text{oper_channel}$	25MHz
115	36,40,44,48	$5000 + 5 * \text{oper_channel}$	20MHz
116	36,44	$5000 + 5 * \text{oper_channel}$	40MHz
117	40,48	$5000 + 5 * \text{oper_channel}$	40MHz
124	149,153,157,161	$5000 + 5 * \text{oper_channel}$	20MHz
125	149,153,157,161,165	$5000 + 5 * \text{oper_channel}$	20MHz
126	149,157	$5000 + 5 * \text{oper_channel}$	40MHz
127	153,161	$5000 + 5 * \text{oper_channel}$	40MHz

3.3.54. ql_wifi_p2p_oper_class_channel_get

该函数用于获取 Wi-Fi P2P 设备的工作频率。

● 函数原型

```
int ql_wifi_p2p_oper_class_channel_get( int *p_oper_class, int *p_oper_channel);
```

● 参数

`p_oper_class`:

[Out] 存放当前 P2P 设备 operating class 的缓存。

`p_oper_channel`:

[Out] 存放当前 P2P 设备 operating channel 的缓存。

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
<code>QL_ERR_INVALID_ARG</code>	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.55. ql_wifi_p2p_ssid_postfix_set

该函数用于设置 Wi-Fi P2P 模式的 SSID 后缀名。

● 函数原型

```
int ql_wifi_p2p_ssid_postfix_set(const char *ssid_postfix);
```

- 参数

ssid_postfix:
[In] SSID 后缀名。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

当 Wi-Fi P2P 为 GO 角色时，具有和 Wi-Fi AP 模式类似的功能，也存在 SSID，其格式为：DIRECT-xx-ssid_postfix，其中 xx 为随机产生的两个字母或者数字，*ssid_postfix* 为 SSID 的后缀名；其他 Wi-Fi STA 模式的设备可以扫描到该 SSID 并进行连接。

3.3.56. ql_wifi_p2p_ssid_postfix_get

该函数用于获取 Wi-Fi P2P 模式的 SSID 后缀名。

- 函数原型

```
int ql_wifi_p2p_ssid_postfix_get(char *ssid_postfix_buf, int buf_len);
```

- 参数

ssid_postfix_buf:
[Out] 指针，指向存放当前 SSID 后缀名的缓存。

buf_len:
[In] 缓存大小。单位：字节。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.57. ql_wifi_p2p_set_enable_status_ind_cb

该函数用于注册 Wi-Fi P2P 模式启用状态回调函数。

- 函数原型

```
int ql_wifi_p2p_set_enable_status_ind_cb(ql_wifi_p2p_enable_status_ind_cb_f cb);
```

- 参数

cb:

[In] Wi-Fi P2P 模式启用状态回调函数。详见第 3.3.57.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.57.1. ql_wifi_p2p_enable_status_ind_cb_f

该回调函数用于处理 Wi-Fi P2P 模式启用状态。

- 函数原型

```
typedef void (*ql_wifi_p2p_enable_status_ind_cb_f)
(QL_WIFI_P2P_ENABLE_STATUS_E pre_status, QL_WIFI_P2P_ENABLE_STATUS_E status);
```

- 参数

pre_status:

[In] Wi-Fi P2P 模式之前的启用状态。详见第 3.3.57.2 章。

status:

[In] Wi-Fi P2P 模式当前的启用状态。详见第 3.3.57.2 章。

- 返回值

无

3.3.57.2. QL_WIFI_P2P_ENABLE_STATUS_E

Wi-Fi P2P 模式的启用状态枚举定义如下：

```
typedef enum
{
    QL_WIFI_P2P_EN_STATUS_DISABLED = 0,
    QL_WIFI_P2P_EN_STATUS_ENABLED,
    QL_WIFI_P2P_EN_STATUS_ERROR,
} QL_WIFI_P2P_ENABLE_STATUS_E;
```

● 成员

成员	描述
<code>QL_WIFI_P2P_EN_STATUS_DISABLED</code>	Wi-Fi P2P 模式已禁用
<code>QL_WIFI_P2P_EN_STATUS_ENABLED</code>	Wi-Fi P2P 模式已启用
<code>QL_WIFI_P2P_EN_STATUS_ERROR</code>	Wi-Fi P2P 模式启用出错

3.3.58. ql_wifi_p2p_enable

该函数用于启用 Wi-Fi P2P 功能。

● 函数原型

```
int ql_wifi_p2p_enable (void);
```

● 参数

无

● 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_NOT_INIT</code>	函数执行失败，未初始化 Wi-Fi 服务；
<code>QL_ERR_SERVICE_ABORT</code>	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <code>ql_type.h</code> 。

3.3.59. ql_wifi_p2p_disable

该函数用于禁用 Wi-Fi P2P 功能。

● 函数原型

```
int ql_wifi_p2p_disable (void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.60. ql_wifi_p2p_set_dev_found_ind_cb

该函数用于注册 P2P 设备扫描信息回调函数。

- 函数原型

```
int ql_wifi_p2p_set_dev_found_ind_cb(ql_wifi_p2p_dev_found_ind_cb_f cb);
```

- 参数

cb:

[In] P2P 设备扫描信息回调函数；详见第 3.3.60.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.60.1. ql_wifi_p2p_dev_found_ind_cb_f

该回调函数用于处理 P2P 设备扫描信息。

- 函数原型

```
typedef void (*ql_wifi_p2p_dev_found_ind_cb_f)(ql_wifi_p2p_found_dev_info_t *p_msg);
```

- 参数

p_msg:

[In] P2P 设备扫描信息。详见第 3.3.60.2 章。

- 返回值

无

3.3.60.2. ql_wifi_p2p_found_dev_info_t

P2P 设备扫描信息结构体定义如下：

```
typedef struct
{
    int is_found
    char macaddr[18];
    char dev_type[32];
    char dev_name[32];
} ql_wifi_p2p_found_dev_info_t;
```

- 参数

类型	参数	描述
int	is_found	P2P 设备扫描结果。
		1 该 P2P 设备已扫描到，可连接该 P2P 设备； 0 该 P2P 设备已丢失，不可连接该 P2P 设备，需重新扫描。
char	macaddr	P2P 设备 MAC 地址
char	dev_type	P2P 设备类型
char	dev_name	P2P 设备名称

3.3.61. ql_wifi_p2p_dev_find_start

该函数用于开始扫描周围的 P2P 设备。

- 函数原型

```
int ql_wifi_p2p_dev_find_start (void);
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功

QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.62. ql_wifi_p2p_dev_find_stop

该函数用于停止扫描周围的 P2P 设备。

- 函数原型

```
int ql_wifi_p2p_dev_find_stop (void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.63. ql_wifi_p2p_get_status

该函数用于获取 Wi-Fi P2P 模式的连接状态信息。

- 函数原型

```
int ql_wifi_p2p_get_status(ql_wifi_p2p_status_t *p_status);
```

- 参数

p_status:

[Out] Wi-Fi P2P 模式的连接状态信息。详见第 3.3.63.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
QL_ERR_INVALID_ARG	函数执行失败，无效参数；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.63.1. ql_wifi_p2p_status_t

Wi-Fi P2P 模式的连接状态信息结构体定义如下：

```
typedef struct
{
    QL_WIFI_P2P_STATUS_E status;
    QL_WIFI_P2P_ROLE_E role;
    int freq;
    char ssid[33];
    char passwd[64];
    char pin_code[9];
    char ifname[32];
    uint8_t has_addr;
    ql_net_addr_t addr;
    uint8_t has_addr6;
    ql_net_addr6_t addr6;
    QL_WIFI_REASON_CODE_E reason_code;
    QL_WIFI_SUB_REASON_CODE_E sub_reason_code;
} ql_wifi_p2p_status_t;
```

● 参数

类型	参数	描述
QL_WIFI_P2P_STATUS_E	status	Wi-Fi P2P 模式连接状态；详见第 3.3.62.2 章。
QL_WIFI_P2P_ROLE_E	role	Wi-Fi P2P 模式的角色；详见第 3.3.62.3 章。
int	freq	Wi-Fi P2P 模式的工作频率
char	ssid	Wi-Fi P2P 模式的 SSID（针对 GO 角色）
char	passwd	Wi-Fi P2P 模式的 SSID 密码（针对 GO 角色）
char	pin_code	Wi-Fi P2P 模式使用 PIN 方式主动连接对端 P2P 设备时，对端设备需要输入 pin_code 进行验证。
char	ifname	Wi-Fi P2P 模式 Linux 网络接口名
uint8_t	has_addr	是否为 IPv4 地址（针对 Client 角色）。 1 是 0 否
ql_net_addr_t	addr	IPv4 地址, ql_net_addr_t 定义在 API 头

		文件 <i>ql_net_common.h</i> 中。(针对 Client 角色)
uint8_t	has_addr6	是否为 IPv6 地址 (针对 Client 角色)。 1 是 0 否
ql_net_addr6_t	addr6	IPv6 地址, <i>ql_net_addr6_t</i> 定义在 API 头文件 <i>ql_net_common.h</i> 中。(针对 Client 角色)
QL_WIFI_REASON_CODE_E	reason_code	Wi-Fi P2P 模式连接失败原因码, <i>QL_WIFI_REASON_CODE_E</i> 定义在 API 头文件 <i>ql_wifi_common.h</i> 中。
QL_WIFI_SUB_REASON_CODE_E	sub_reason_code	Wi-Fi P2P 模式连接失败次要原因码, <i>QL_WIFI_SUB_REASON_CODE_E</i> 定义在 API 头文件 <i>ql_wifi_common.h</i> 中。

3.3.63.2. QL_WIFI_P2P_STATUS_E

Wi-Fi P2P 模式的连接状态枚举定义如下:

```
typedef enum
{
    QL_WIFI_P2P_STATUS_IDLE = 0,
    QL_WIFI_P2P_STATUS_CONNECTING,
    QL_WIFI_P2P_STATUS_ASSOCIATED,
    QL_WIFI_P2P_STATUS_CONNECTED,
    QL_WIFI_P2P_STATUS_DISCONNECTED,
    QL_WIFI_P2P_STATUS_ERROR
} QL_WIFI_P2P_STATUS_E;
```

● 成员

成员	描述
<i>QL_WIFI_P2P_STATUS_IDLE</i>	还未开始和对端 P2P 设备进行连接
<i>QL_WIFI_P2P_STATUS_CONNECTING</i>	正在连接对端 P2P 设备, 包括 P2P 角色的协商。
<i>QL_WIFI_P2P_STATUS_ASSOCIATED</i>	如果 <i>CONNECTING</i> 过程中协商的是 Client 角色, 则该状态表示已经连上对端 P2P 设备, 但未分配 IP 地址; 如果为 GO 角色, 则没有该状态。
<i>QL_WIFI_P2P_STATUS_CONNECTED</i>	已经连上对端 P2P 设备, 且 IP 地址已分配。
<i>QL_WIFI_P2P_STATUS_DISCONNECTED</i>	P2P 连接断开

QL_WIFI_P2P_STATUS_ERROR	P2P 连接出错
--------------------------	----------

3.3.63.3. QL_WIFI_P2P_ROLE_E

Wi-Fi P2P 模式的角色枚举定义如下：

```
typedef enum
{
    QL_WIFI_P2P_ROLE_NONE = 0,
    QL_WIFI_P2P_ROLE_CLI,
    QL_WIFI_P2P_ROLE_GO
} QL_WIFI_P2P_ROLE_E;
```

● 成员

成员	描述
QL_WIFI_P2P_ROLE_NONE	还未开始 P2P 角色协商或者协商失败
QL_WIFI_P2P_ROLE_CLI	协商成功，且为 P2P Client 角色。
QL_WIFI_P2P_ROLE_GO	协商成功，且为 P2P GO 角色。

3.3.64. ql_wifi_p2p_set_status_ind_cb

该函数用于注册 Wi-Fi P2P 模式连接状态回调函数。

● 函数原型

```
int ql_wifi_p2p_set_status_ind_cb(ql_wifi_p2p_status_ind_cb_f cb);
```

● 参数

cb:

[In] Wi-Fi P2P 模式连接状态回调函数。详见第 3.3.64.1 章。

● 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 ql_type.h。

3.3.64.1. ql_wifi_p2p_status_ind_cb_f

该回调函数用于处理 Wi-Fi P2P 模式连接状态。

- 函数原型

```
typedef void(*ql_wifi_p2p_status_ind_cb_f)(QL_WIFI_P2P_STATUS_E pre_status,
ql_wifi_p2p_status_t *p_msg);
```

- 参数

pre_status:

[In] Wi-Fi P2P 模式之前的连接状态；详见第 3.3.63.2 章。

p_msg:

[In] Wi-Fi P2P 模式当前的连接状态信息；详见第 3.3.63.1 章。

- 返回值

无

3.3.65. ql_wifi_p2p_set_peer_dev_req_ind_cb

该函数用于注册对端 P2P 设备连接请求回调函数。

- 函数原型

```
int ql_wifi_p2p_set_peer_dev_req_ind_cb(ql_wifi_p2p_peer_dev_req_ind_cb_f cb);
```

- 参数

cb:

[In] 对端 P2P 设备连接请求回调函数；详见第 3.3.64.1 章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_NOT_INIT	函数执行失败，未初始化 Wi-Fi 服务；
QL_ERR_SERVICE_ABORT	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.65.1. ql_wifi_p2p_peer_dev_req_ind_cb_f

该回调函数用于处理对端 P2P 设备连接请求。

- 函数原型

```
typedef void (*ql_wifi_p2p_peer_dev_req_ind_cb_f)(ql_wifi_p2p_req_peer_dev_info_t *pmsg);
```

- 参数

pmsg:

[In] 对端 P2P 设备信息；详见第 3.3.65.2 章。

- 返回值

无

3.3.65.2. ql_wifi_p2p_req_peer_dev_info_t

对端 P2P 设备信息结构体定义如下：

```
typedef struct
{
    char macaddr[18];
} ql_wifi_p2p_req_peer_dev_info_t;
```

- 参数

类型	参数	描述
char	<i>macaddr</i>	对端 P2P 设备 MAC 地址。

3.3.66. ql_wifi_p2p_connect

该函数用于连接对端 P2P 设备。

- 函数原型

```
int ql_wifi_p2p_connect (const char *macaddr, const char *method, int go_intent);
```

- 参数

macaddr:

[In] 对端 P2P 设备的 MAC 地址。

method:

[In] P2P 连接方式。取值：*pbcc* 或 *pin*。当本地 P2P 设备主动使用 *pin* 方式发起连接，本地设备会产生一个 PIN 码，对端设备连接时，要输入这个 PIN 码；本地设备使用 *pbcc* 方式发起连接时，对端设备可以直接连。当对端设备使用 *pin* 方式主动连接本地 P2P 设备时，则该参数为对端 P2P 设备产生的 PIN 码。

go_intent:

[In] 成为 GO 角色的渴望程度。类型：整数；取值范围：0~15。值越大，则表示越渴望成为 GO 角色，一般情况下，要想成为 Client 角色，则该参数设置为 0，要想成为 GO 角色，则该参数设置为 15。

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化 Wi-Fi 服务；
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.67. ql_wifi_p2p_disconnect

该函数用于断开对端 P2P 设备。

- 函数原型

```
int ql_wifi_p2p_disconnect(void);
```

- 参数

无

- 返回值

<i>QL_ERR_OK</i>	函数执行成功
<i>QL_ERR_NOT_INIT</i>	函数执行失败，未初始化 Wi-Fi 服务；
<i>QL_ERR_SERVICE_ABORT</i>	函数执行失败，Wi-Fi 服务出错；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

4 示例

Wi-Fi API 示例代码请参考 `sample/test_sdk_api/m_wifi.c`，本章主要介绍如何使用 Wi-Fi API 设置 STA 模式、单 AP 模式以及 P2P 模式。

4.1. 设置 STA 模式

```

/** 定义 Wi-Fi 服务最大初始化次数 */
#define WIFI_INIT_MAX_TRY_CNT 50

int main()
{
    int ret = QL_ERR_OK;
    int wifi_is_init = 0;
    int wifi_init_try_cnt = 0;
    ql_wifi_sta_auth_t auth;

    /** Wi-Fi 服务初始化 */
    while (WIFI_INIT_MAX_TRY_CNT > wifi_init_try_cnt)
    {
        ret = ql_wifi_init();
        if (QL_ERR_SERVICE_NOT_READY == ret)
        {
            printf("wifi service is not ready, try again, try count = %d\n", (wifi_init_try_cnt +
1));
            usleep(200 * 1000);
            wifi_init_try_cnt++;
        }
        else if (QL_ERR_OK == ret)
        {
            printf("Succeeded to initialize wifi service\n");
            wifi_is_init = 1;
            break;
        }
        else
        {
            printf("Failed to initialize wifi service, ret = %d\n", ret);

```

```

        break;
    }
}

if (1 != wifi_is_init)
{
    printf("Failed to initialize wifi service\n");
    return -1;
}

/**启用 Wi-Fi */
ret = ql_wifi_enable();
if (QL_ERR_OK != ret)
{
    printf("Failed to enable wifi, ret = %d", ret);
    return -1;
}

/** 设置 Wi-Fi 工作模式为 STA 模式 */
ret = ql_wifi_work_mode_set(QL_WIFI_WORK_MODE_STA);
if (QL_ERR_OK != ret)
{
    printf("Failed to set wifi work mode, ret = %d", ret);
    return -1;
}

/** 设置 SSID */
ret = ql_wifi_sta_ssid_set("Quectel-Hf");
if (QL_ERR_OK != ret)
{
    printf("Failed to set ssid, ret = %d", ret);
    return -1;
}

/** 设置 STA 模式安全认证 */
memset(&auth, 0, sizeof(ql_wifi_sta_auth_t));
auth.auth = QL_WIFI_AUTH_WPA_WPA2_PSK_BOTH;
auth.wpa_psk.pairwise = QL_WIFI_AUTH_WPA_PAIRWISE_AUTO;
strncpy (auth.wpa_psk.passwd, "*quectel-i-hf*", sizeof(auth.wpa_psk.passwd) - 1);

ret = ql_wifi_sta_auth_set(&auth);
if (QL_ERR_OK != ret)
{
    printf("Failed to set wifi auth, ret = %d", ret);
}

```

```

        return -1;
    }

    /** 开启 STA 模式 */
    ret = ql_wifi_sta_start();
    if (QL_ERR_OK != ret)
    {
        printf("Failed to start sta, ret = %d", ret);
        ql_wifi_disable();
        return -1;
    }

    return 0;
}

```

4.2. 设置单 AP 模式

```

/** 定义 Wi-Fi 服务最大初始化次数 */
#define WIFI_INIT_MAX_TRY_CNT 50

int main()
{
    int ret = QL_ERR_OK;
    int wifi_is_init = 0;
    int wifi_init_try_cnt = 0;
    ql_wifi_ap_auth_t auth;

    /** Wi-Fi 服务初始化 */
    while (WIFI_INIT_MAX_TRY_CNT > wifi_init_try_cnt)
    {
        ret = ql_wifi_init();
        if (QL_ERR_SERVICE_NOT_READY == ret)
        {
            printf("wifi service is not ready, try again, try count = %d\n", (wifi_init_try_cnt +
1));
            usleep(200 * 1000);
            wifi_init_try_cnt++;
        }
        else if (QL_ERR_OK == ret)
        {
            printf("Succeeded to initialize wifi service\n");

```

```

        wifi_is_init = 1;
        break;
    }
    else
    {
        printf("Failed to initialize wifi service, ret = %d\n", ret);
        break;
    }
}

if (1 != wifi_is_init)
{
    printf("Failed to initialize wifi service\n");
    return -1;
}

/** 启用 Wi-Fi */
ret = ql_wifi_enable();
if (QL_ERR_OK != ret)
{
    printf("Failed to enable wifi, ret = %d", ret);
    return -1;
}

/** 设置 Wi-Fi 工作模式为单 AP 模式 */
ret = ql_wifi_work_mode_set(QL_WIFI_WORK_MODE_AP0);
if (QL_ERR_OK != ret)
{
    printf("Failed to set wifi work mode, ret = %d", ret);
    return -1;
}

/** 设置信道，带宽等配置此处省略 */
.....

/** 设置 SSID */
ret = ql_wifi_ap_ssid_set("Quectel-AP0");
if (QL_ERR_OK != ret)
{
    printf("Failed to set ssid, ret = %d", ret);
    return -1;
}

/** 设置 AP 模式安全认证 */

```

```

memset(&auth, 0, sizeof(ql_wifi_ap_auth_t));
auth.auth = QL_WIFI_AUTH_WPA_WPA2_PSK_BOTH;
auth.wpa_psk.pairwise = QL_WIFI_AUTH_WPA_PAIRWISE_AUTO;
strncpy (auth.wpa_psk.passwd, "123456789", sizeof(auth.wpa_psk.passwd) - 1);

ret = ql_wifi_ap_auth_set(QL_WIFI_AP_INDEX_AP0, &auth);
if (QL_ERR_OK != ret)
{
    printf("Failed to set wifi auth, ret = %d", ret);
    return -1;
}

/** 开启 AP 模式 */
ret = ql_wifi_ap_start(QL_WIFI_AP_INDEX_AP0);
if (QL_ERR_OK != ret)
{
    printf("Failed to start ap[%d], ret = %d", QL_WIFI_AP_INDEX_AP0, ret);
    ql_wifi_disable();
    return -1;
}

return 0;
}

```

4.3. Wi-Fi P2P 模式

```

/** 定义 Wi-Fi 服务最大初始化次数 */
#define WIFI_INIT_MAX_TRY_CNT 50

void wifi_p2p_dev_found_ind_cb( ql_wifi_p2p_found_dev_info_t *p_msg)
{
    /** 将 p2p 设备信息发给主进程，可以使用 socketpair */
}

int main()
{
    int ret = QL_ERR_OK;
    int wifi_is_init = 0;
    int wifi_init_try_cnt = 0;
    ql_wifi_sta_auth_t auth;

```

```

/** Wi-Fi 服务初始化 */
while (WIFI_INIT_MAX_TRY_CNT > wifi_init_try_cnt)
{
    ret = ql_wifi_init();
    if (QL_ERR_SERVICE_NOT_READY == ret)
    {
        printf("wifi service is not ready, try again, try count = %d\n", (wifi_init_try_cnt +
1));
        usleep(200 * 1000);
        wifi_init_try_cnt++;
    }
    else if (QL_ERR_OK == ret)
    {
        printf("Succeeded to initialize wifi service\n");
        wifi_is_init = 1;
        break;
    }
    else
    {
        printf("Failed to initialize wifi service, ret = %d\n", ret);
        break;
    }
}

if (1 != wifi_is_init)
{
    printf("Failed to initialize wifi service\n");
    return -1;
}

/** 启用 Wi-Fi */
ret = ql_wifi_enable();
if (QL_ERR_OK != ret)
{
    printf("Failed to enable wifi, ret = %d", ret);
    return -1;
}

/** 设置 P2P 设备名称 */
ret = ql_wifi_p2p_dev_name_set("Quectel-P2P");
if (QL_ERR_OK != ret)
{
    printf("Failed to set ssid, ret = %d", ret);
    return -1;
}

```

```

    }

    其他配置接口省略....

    ret = ql_wifi_p2p_enable();
    if (QL_ERR_OK != ret)
    {
        printf("Failed to enable p2p, ret = %d", ret);
        ql_wifi_disable();
        return -1;
    }

    ret = ql_wifi_p2p_set_dev_found_ind_cb(p2p_dev_found_ind_cb);
    if (QL_ERR_OK != ret)
    {
        printf("Failed to set dev found ind cb, ret = %d", ret);
        ql_wifi_p2p_disable();
        ql_wifi_disable();
        return -1;
    }

    ret = ql_wifi_p2p_dev_find_start();
    if (QL_ERR_OK != ret)
    {
        printf("Failed to start finding p2p dev, ret = %d", ret);
        ql_wifi_p2p_disable();
        ql_wifi_disable();
        return -1;
    }

    /** 等待 wifi_p2p_dev_found_ind_cb 上报对端 P2P 设备信息， 可以使用 select 机制 */

    /** 并连接对端 P2P 设备 */
    ret = ql_wifi_p2p_connect("对端设备 MAC 地址", "pbc", 15);
    if (QL_ERR_OK != ret)
    {
        printf("Failed to connect p2p, ret = %d", ret);
        ql_wifi_p2p_disable();
        ql_wifi_disable();
        return -1;
    }

    return 0;
}

```

5 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_AG35-CET_QuecOpen(SDK)_快速开发指导

表 3：术语缩写

缩写	英文全称	中文全称
AES	Advanced Encryption Standard	高级加密标准
AP	Access Point	接入点
API	Application Programming Interface	应用程序编程接口
BSSID	Basic Service Set Identifier	基本服务标识符
ESSID	Extended Service Set Identifier	扩展服务集标识符
GO	Group Owner	组所有者
IoV	Internet of Vehicles	车联网
IPv4	Internet Protocol version 4	第 4 版互联网协议
IPv6	Internet Protocol version 6	第 6 版互联网协议
MAC	Media Access Control (Address)	媒体访问控制（位址）
PSK	Pre-Shared Key	预共享的密钥
P2P	Peer to Peer	点对点
SDK	Software Development Kit	软件开发工具包
SSID	Service Set Identifier	服务集标识符

STA	Station	站点
TKIP	Temporal Key Integrity Protocol	临时密钥完整性协议
WEP	Wired Equivalent Privacy	有线等效保密
WLAN	Wireless Local Area Networks	无线局域网
WPA	Wi-Fi Protected Access	Wi-Fi 网络安全接入
WPS	Wi-Fi Protected Setup	Wi-Fi 保护设置
WSC	Wi-Fi Simple Configuration	Wi-Fi 简化配置

Preliminary
Quectel Confidential